建設業BPO詳細設計

> **スコープ**: 公共工事 + 民間工事、土木中心（建築は Phase B+）
> **パイロット**: 総合建設 + 専門工事（両方）
> **制約**: 積算ソフト・単価DBアクセスなし → 自前構築が必要
> **目標**: 5週間で建設業BPO全8モジュール稼働

---

0. エグゼクティブサマリー

なぜ建設業BPOか

```
建設投資総額: 80兆円/年（2026年度予測）
建設DX市場: 586億円 → 1,250億円（2030年）
i-Construction 2.0: 2040年までに生産性1.5倍（国策）

建設業の中小企業（10-300名）は：
  ・積算できる人がいない / 引退していく
  ・安全書類に毎現場で何十時間も費やしている
  ・出来高管理がExcel地獄
  ・月末に経理が死んでいる
 → 全部AIで代行できる = BPOコア¥25万 + 追加モジュール¥10万/個の価値
```

8モジュール全体像

```
┌──────────────────────────────────────────────────────────────┐
│              建設業BPO（BPOコア¥250,000 + 追加モジュール¥100,000/個）                  │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  ★ Phase A（Week 1–2）: 最優先                                 │
│  ┌──────────────────┐  ┌──────────────────────────┐          │
│  │ ① 積算AI          │  │ ② 安全書類自動生成          │          │
│  │ 図面→数量→単価→内訳 │  │ 作業員名簿・施工体制台帳     │          │
│  │ ¥10–30万/月の価値   │  │ ¥3–5万/月の価値             │          │
│  └──────────────────┘  └──────────────────────────┘          │
│                                                              │
│  Phase B（Week 3）:                                            │
│  ┌──────────────────┐  ┌──────────────────────────┐          │
│  │ ③ 出来高・請求書     │  │ ④ 原価管理レポート          │          │
│  │ 出来高計算→請求自動化 │  │ 予実分析→赤字工事予測       │          │
│  └──────────────────┘  └──────────────────────────┘          │
│                                                              │
│  Phase C（Week 4）:                                            │
│  ┌──────────────────┐  ┌──────────────────────────┐          │
│  │ ⑤ 施工計画書        │  │ ⑥ 工事写真整理              │          │
│  │ テンプレ→AI生成      │  │ AI分類+黒板OCR+電子納品     │          │
│  └──────────────────┘  └──────────────────────────┘          │
│                                                              │
│  Phase D（Week 5）:                                            │
│  ┌──────────────────┐  ┌──────────────────────────┐          │
│  │ ⑦ 下請管理          │  │ ⑧ 許可更新・経審サポート     │          │
│  │ 評価+発注+支払一元化 │  │ 書類自動生成+期限管理       │          │
│  └──────────────────┘  └──────────────────────────┘          │
│                                                              │
│  横断:                                                         │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ SaaS連携層（ANDPAD API / CSV取込 / Excel入出力）       ||    │
│  └──────────────────────────────────────────────────────┘    │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ 単価DB（公共工事設計労務単価 + 市場単価 + 自社実績）     ||    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

---

1. 積算AI（★キラーフィーチャー）

1.1 問題の本質

```
積算の現状:
  1. 図面を読む（設計図書、数量計算書、仕様書）
  2. 数量を拾い出す（長さ、面積、体積、個数）
  3. 単価を選定する（設計単価、市場単価、見積単価）
  4. 歩掛を適用する（標準歩掛 × 補正係数）
  5. 諸経費を計算する（共通仮設費、現場管理費、一般管理費等）
  6. 内訳書を作成する

  → 熟練者が1案件3-7日。ミスれば赤字受注で数百万〜数千万の損失
  → 熟練積算士の平均年齢55歳以上。10年後に半減する
  → 中小建設会社では社長自身がやっている or 外注（1件5-20万円）
```

1.2 シャチョツーの積算AI — 3段階アプローチ

単価DBがない制約を逆手に取る: 「AI積算アシスタント」から始めて「完全自動積算」に進化する。

■ Level 1: 拾い出しAI ✅ 実装・実データ検証済み（2026-03-18）
入力: 設計書PDF（国交省設計内訳書形式）
処理: PDFテキスト前処理（単位行→数量行パターン検出）→ ルールベース工種推定
※ LLM不要。前処理のみで構造化抽出。LLMはフォールバック時のみ
出力: 構造化された数量一覧（工種/種別/細別/規格/数量/単位/単価）

実績（2026-03-18 実データ検証）:
- 15件のPDF処理、12件成功（80%。失敗3件はスキャン画像PDF）
- 合計356件の数量を自動抽出
- 工種分類精度: 96.3%（18カテゴリのキーワードルール）
- 単価取得率: 100%（設計書から直接抽出）
- 処理時間: 1.6秒/PDF（API呼び出し0回、コスト¥0）

価値: 拾い出し作業の90%削減（3日→数秒）
制約: PDFテキスト形式のみ対応。スキャン画像はOCR未対応（Phase 2+）

■ Level 2: 単価推定AI ✅ 実装・学習データ投入済み（2026-03-18）
入力: Level 1の数量一覧 + 地域 + 工事種別
処理:
① 設計書から直接取得（confidence=0.95）← ★新手法。設計書に単価が記載されている場合
② 自社の過去積算実績DB（加重平均 + 動的confidence）
③ 公共工事設計労務単価（PDF→構造化済み）
④ LLMによる単価推定（工種・地域・時期から推定）← 候補がない場合のみ
出力: 単価候補（推定値 + 信頼度）付きの内訳書

実績:
- unit_price_masterに718件の単価データを初期投入済み
- 地域: 東京都272件、岐阜県207件、長野県149件、神奈川県90件
- 年度: 2023-2025年度
- 加重平均（直近重視）+ 動的confidence（実績件数・地域一致・精度ペナルティ）

価値: 積算全体の80%削減（3日→半日）
制約: 推定単価は必ず人間が確認

■ Level 3: 学習型積算（Phase 2+）
入力: 過去の積算データN社分（匿名化）
処理:
① 企業固有の単価傾向学習
② 同業種・同地域の市場単価推定
③ 落札率予測（公共工事）
出力: 高精度な自動積算 + 受注戦略提案

基盤状況:
- フィードバック学習ループ（Sprint 1-4）全実装済み
- unit_price_master蓄積、accuracy_rate記録、used_count追跡
- 正規化辞書（term_normalization）自動登録
- Trust Scorer（承認率ベースの自律度レベル管理）
- llm_call_logs（成功・失敗両方の記録）

```
    価値: 「積算士不要」レベル
    → これがNetwork Effect（Moat）
```

1.3 積算AIの技術設計

データモデル

```
-- 積算プロジェクト
CREATE TABLE estimation_projects (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  name TEXT NOT NULL,                    -- 工事名
  project_type TEXT NOT NULL,            -- public_civil / public_building /
private_civil / private_building
  region TEXT NOT NULL,                  -- 都道府県
  municipality TEXT,                     -- 市区町村
  fiscal_year INTEGER NOT NULL,          -- 年度（単価基準年度）
  client_name TEXT,                      -- 発注者名
  design_amount BIGINT,                  -- 設計金額（円）※公共工事の場合
  estimated_amount BIGINT,              -- 積算金額（円）
  status TEXT DEFAULT 'draft',           -- draft / in_progress / review / submitted /
won / lost
  overhead_rates JSONB DEFAULT '{}',     -- 諸経費率 {"common_temporary": 0.xx,
"site_management": 0.xx, "general_admin": 0.xx}
  metadata JSONB DEFAULT '{}',           -- 工期、施工条件等
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 積算明細（数量 × 単価）
CREATE TABLE estimation_items (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID NOT NULL REFERENCES estimation_projects(id) ON DELETE CASCADE,
  company_id UUID NOT NULL REFERENCES companies(id),
  sort_order INTEGER NOT NULL,
  category TEXT NOT NULL,               -- 工種（土工、コンクリート工、鋼構造物工 等）
  subcategory TEXT,                      -- 種別（掘削工、埋戻工 等）
  detail TEXT,                           -- 細別（バックホウ掘削、人力掘削 等）
  specification TEXT,                    -- 規格（0.8m3級、N=10回 等）
  quantity DECIMAL(15,3) NOT NULL,       -- 数量
  unit TEXT NOT NULL,                    -- 単位（m3, m2, m, 本, 式 等）
  unit_price DECIMAL(15,2),             -- 単価（円）
  amount BIGINT GENERATED ALWAYS AS (ROUND(quantity * COALESCE(unit_price, 0))) STORED,
  price_source TEXT,                     -- manual / past_record / labor_rate /
market_price / ai_estimated
  price_confidence DECIMAL(3,2),        -- 単価信頼度 0.00-1.00
  source_document TEXT,                  -- 拾い出し元（図面ページ、数量計算書行番号等）
  notes TEXT,                            -- 備考
  created_at TIMESTAMPTZ DEFAULT now()
);

-- 単価マスタ（自社実績ベース。使うほど育つ）
CREATE TABLE unit_price_master (
```

```
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  category TEXT NOT NULL,                -- 工種
  subcategory TEXT,                      -- 種別
  detail TEXT,                           -- 細別
  specification TEXT,                    -- 規格
  unit TEXT NOT NULL,                    -- 単位
  unit_price DECIMAL(15,2) NOT NULL,     -- 単価
  price_type TEXT NOT NULL,              -- labor / material / machine / composite
  region TEXT,                           -- 地域
  year INTEGER,                          -- 年度
  source TEXT NOT NULL,                  -- manual_input / past_estimation /
public_labor_rate / market_survey
  source_detail TEXT,                    -- ソース詳細（参照元の工事名等）
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 公共工事設計労務単価（全国共通、年度更新）
CREATE TABLE public_labor_rates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  fiscal_year INTEGER NOT NULL,          -- 年度
  region TEXT NOT NULL,                  -- 都道府県
  occupation TEXT NOT NULL,              -- 職種（普通作業員、特殊作業員、軽作業員、とび工、鉄筋
工...）
  daily_rate INTEGER NOT NULL,           -- 日額（円）
  source_url TEXT,                       -- 出典URL
  created_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE(fiscal_year, region, occupation)
);

-- 積算テンプレート（工事種別ごとの標準工種構成）
CREATE TABLE estimation_templates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,                    -- テンプレート名（道路改良工事、河川護岸工事 等）
  project_type TEXT NOT NULL,            -- public_civil / private_civil 等
  category TEXT NOT NULL,                -- 土木 / 建築
  items JSONB NOT NULL,                  -- 標準工種構成 [{category, subcategory, detail,
specification, unit, notes}]
  overhead_defaults JSONB NOT NULL,      -- デフォルト諸経費率
  created_at TIMESTAMPTZ DEFAULT now()
);

-- RLS
ALTER TABLE estimation_projects ENABLE ROW LEVEL SECURITY;
ALTER TABLE estimation_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE unit_price_master ENABLE ROW LEVEL SECURITY;
-- public_labor_rates と estimation_templates は全社共通（RLS不要、読み取り専用）
```

積算AIパイプライン

```
# workers/bpo/construction/estimator.py

class EstimationPipeline:
    """
    積算AIパイプライン

    Step 1: 図面・数量計算書の取り込み（Document AI + LLM）
    Step 2: 数量の構造化抽出
    Step 3: 単価の推定・候補表示
    Step 4: 内訳書生成
    Step 5: 諸経費計算
    """

    async def ingest_documents(
        self,
        project_id: str,
        files: list[UploadFile],
    ) -> IngestionResult:
        """
        図面PDF/Excel/画像を取り込み、構造化する

        対応形式:
        - 数量計算書（Excel）→ 最も精度が高い。列マッピングで構造化
        - 設計書PDF（テキスト）→ LLMで工種/数量/単位を抽出
        - 図面PDF/画像 → Document AI でOCR → LLMで構造化
        - 既存積算ソフトの出力CSV/Excel → フォーマット自動判定
        """

    async def extract_quantities(
        self,
        project_id: str,
        document_id: str,
    ) -> list[EstimationItem]:
        """
        ドキュメントから数量を構造化抽出

        LLMプロンプト:
        「以下の建設工事の設計書/数量計算書から、
         工種・種別・細別・規格・数量・単位を抽出してください。
         土木工事の標準的な分類体系に従ってください。」

        出力: [{category, subcategory, detail, spec, quantity, unit, source_ref}]
        """

    async def suggest_unit_prices(
        self,
        project_id: str,
        items: list[EstimationItem],
    ) -> list[EstimationItemWithPrice]:
```

```
        """
        各項目に単価候補を付与

        優先順位:
        1. 自社過去実績（同一工種・同一地域・直近2年以内）→ 信頼度0.9
        2. 公共工事設計労務単価（労務費の場合）→ 信頼度0.95
        3. 自社過去実績（類似工種・同一地域）→ 信頼度0.7
        4. LLM推定（工種・地域・時期から推定）→ 信頼度0.3-0.5
        5. 候補なし → ユーザーに手動入力を求める
        """

    async def calculate_overhead(
        self,
        project_id: str,
        direct_cost: int,
        project_type: str,   # public_civil / private
        project_scale: str,  # small / medium / large
    ) -> OverheadBreakdown:
        """
        諸経費を計算

        公共土木の場合（国交省基準）:
          共通仮設費 = 直接工事費 × 率（工事規模による）
          現場管理費 = (直接工事費 + 共通仮設費) × 率
          一般管理費等 = (直接工事費 + 共通仮設費 + 現場管理費) × 率

        民間の場合:
          諸経費率 = 会社設定（デフォルト27%）
        """

    async def generate_breakdown(
        self,
        project_id: str,
        format: str = "excel",  # excel / pdf / csv
    ) -> bytes:
        """
        内訳書を生成

        出力形式:
        - 公共工事: 工事費内訳書（国交省標準書式）
        - 民間工事: 見積書（自社書式）
        """

    async def learn_from_result(
        self,
        project_id: str,
        actual_prices: list[dict],  # ユーザーが修正した単価
    ) -> None:
        """
        ユーザーの修正から学習
        - 修正された単価を unit_price_master に反映
        - 信頼度を更新
```

```
        - 次回以降の推定精度を向上
        """
```

単価DBの構築戦略（単価DBアクセスなしの制約対応）

```
■ Phase 1（Week 1-2）: 最小限の単価データ

  ① 公共工事設計労務単価（国交省PDF → 手動/OCR でDB化）
     - 51職種 × 47都道府県 × 年度 = 約2,400レコード/年
     - 直近3年分で約7,200レコード
     - これは公開データなので無償利用可能

  ② 積算テンプレート（標準的な工種構成）
     - 道路改良、河川護岸、下水道、舗装、橋梁補修 等
     - 10テンプレート × 平均30工種 = 約300レコード
     - 一般的な知識としてLLMで生成可能

  ③ ユーザー入力
     - 初回は手動入力だが、2回目以降は過去実績から自動候補
     - 使うほど自社単価DBが育つ

■ Phase 2（Week 3-4）: 単価推定の精度向上

  ④ 過去積算データの蓄積
     - ユーザーが過去の積算書（Excel）をアップロード → 単価マスタに反映
     - 「過去3年分の積算書を取り込んでください」→ 自社単価DBが一気に充実

  ⑤ LLM単価推定
     - 工種 + 地域 + 時期 → LLMが一般的な単価レンジを推定
     - 信頼度を明示し、必ず人間が確認
     - 「この単価は推定です（信頼度40%）。確認してください」

■ Phase 3（Phase 2+）: ネットワーク効果

  ⑥ 匿名化クロステナント単価
     - N社の単価データを匿名化して集約
     - 同一地域・同一工種の市場単価を統計的に推定
     - 「同業他社の直近6ヶ月の平均単価: ¥XX,XXX」

  ⑦ 建設物価データ連携（有償ライセンス）
     - 月額10-30万円程度のデータライセンス費用
     - 顧客数が増えて採算が合う時点で契約
     - 建設物価調査会 or 経済調査会のデータファイル
```

1.4 積算AIのUI設計

```
■ 積算プロジェクト一覧画面
  /bpo/estimation
  - プロジェクト一覧（工事名、発注者、状況、金額）
  - 新規作成ボタン
  - フィルター（状態、工事種別）

■ 積算プロジェクト作成
  /bpo/estimation/new
  Step 1: 基本情報入力
    - 工事名、工事種別（公共土木/民間等）、地域、年度
    - テンプレート選択（道路改良、河川護岸等）
  Step 2: 図面・設計書アップロード
    - ドラッグ&ドロップで複数ファイル
    - 対応形式: PDF, Excel, 画像(JPG/PNG)
    - アップロード後、AI解析中の進捗表示
  Step 3: 数量確認・編集
    - AI抽出結果をテーブル表示
    - 行ごとに編集可能（追加・削除・修正）
    - 信頼度が低い項目はハイライト
    - 「抽出元を表示」で元図面の該当箇所を表示
  Step 4: 単価設定
    - 各項目に単価候補を表示（ソース・信頼度付き）
    - ユーザーが選択 or 手動入力
    - 「過去実績から候補」「AI推定」のタブ切り替え
  Step 5: 諸経費・最終確認
    - 諸経費率の設定（デフォルト値 or カスタム）
    - 工事費総括表の表示
    - 利益率シミュレーション
  Step 6: 出力
    - Excel/PDF ダウンロード
    - 過去実績DBに自動保存

■ 単価マスタ管理
  /bpo/estimation/prices
  - 自社単価一覧（検索・フィルタ）
  - 過去積算書一括取り込み
  - 公共工事設計労務単価閲覧（年度・地域別）
```

2. 安全書類自動生成

2.1 問題の本質

```
中小建設会社の安全書類の現状:
  - 新しい現場が始まるたびに、ほぼ同じ書類を一から作り直す
  - 作業員が入れ替わるたびに作業員名簿を更新
  - 元請ごとにフォーマットが微妙に違う
  - 資格証の有効期限管理がExcelかノート
  - グリーンサイトを使っている元請もあれば紙の元請もある

  → 1現場あたり初回20-40時間、更新で月5-10時間
  → 全現場合計で月50-100時間（事務員1人分の工数）
```

2.2 対象書類と自動化レベル

```
■ 完全自動生成（マスタデータから100%自動）
  - 第5号: 作業員名簿 ← 最大の工数削減
  - 第5号別紙: 社会保険加入状況
  - 第9号: 有資格者一覧表
  - 第1号-甲: 下請負業者編成表

■ 半自動生成（テンプレート + AI補完）
  - 第1号: 再下請負通知書
  - 第2号: 施工体制台帳
  - 第3号: 施工体系図
  - 第6号: 工事安全衛生計画書
  - 第7号: 新規入場時等教育実施報告書

■ テンプレート提供のみ
  - 第10号: 持込機械使用届
  - 第11号: 持込機械（電動工具等）使用届
  - 第12号: 火気使用願
```

2.3 データモデル

```
-- 作業員マスタ（会社の全作業員）
CREATE TABLE workers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  last_name TEXT NOT NULL,
  first_name TEXT NOT NULL,
  last_name_kana TEXT,
  first_name_kana TEXT,
  birth_date DATE,
  blood_type TEXT,                        -- A/B/O/AB
  address TEXT,
  phone TEXT,
  hire_date DATE,                         -- 雇入年月日
  experience_years INTEGER,               -- 経験年数
  health_check_date DATE,                 -- 最新健康診断日
  health_check_result TEXT,               -- 結果
  social_insurance JSONB DEFAULT '{}',    -- {health_insurance: {number, provider},
pension: {number}, employment: {number}}
  emergency_contact JSONB DEFAULT '{}',   -- {name, relationship, phone}
  status TEXT DEFAULT 'active',           -- active / inactive / retired
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 資格マスタ
CREATE TABLE worker_qualifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  worker_id UUID NOT NULL REFERENCES workers(id) ON DELETE CASCADE,
  company_id UUID NOT NULL REFERENCES companies(id),
  qualification_type TEXT NOT NULL,        -- license / special_training /
skill_training
  qualification_name TEXT NOT NULL,        -- 資格名（例: 1級土木施工管理技士）
  certificate_number TEXT,                 -- 資格証番号
  issued_date DATE,                        -- 取得日
  expiry_date DATE,                        -- 有効期限（ない場合はNULL）
  issuer TEXT,                             -- 発行機関
  certificate_image_url TEXT,              -- 資格証画像URL
  created_at TIMESTAMPTZ DEFAULT now()
);

-- 現場マスタ
CREATE TABLE construction_sites (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  name TEXT NOT NULL,                      -- 現場名
  address TEXT,                            -- 現場住所
  client_name TEXT,                        -- 元請/発注者名
  contract_amount BIGINT,                  -- 契約金額
  start_date DATE,                         -- 着工日
```

```
  end_date DATE,                          -- 竣工予定日
  site_manager_id UUID REFERENCES workers(id), -- 現場代理人
  safety_officer_id UUID REFERENCES workers(id), -- 安全衛生責任者
  status TEXT DEFAULT 'active',           -- planning / active / completed
  green_file_format TEXT DEFAULT 'zenken', -- zenken（全建統一）/ custom / greensite
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 現場×作業員アサイン
CREATE TABLE site_worker_assignments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  site_id UUID NOT NULL REFERENCES construction_sites(id) ON DELETE CASCADE,
  worker_id UUID NOT NULL REFERENCES workers(id),
  company_id UUID NOT NULL REFERENCES companies(id),
  entry_date DATE NOT NULL,               -- 入場日
  exit_date DATE,                         -- 退場日
  role TEXT,                              -- 職長、作業員等
  entry_education_date DATE,              -- 新規入場者教育実施日
  created_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE(site_id, worker_id, entry_date)
);

-- 生成された安全書類
CREATE TABLE safety_documents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  site_id UUID NOT NULL REFERENCES construction_sites(id),
  company_id UUID NOT NULL REFERENCES companies(id),
  document_type TEXT NOT NULL,            -- worker_roster / org_chart /
qualification_list 等
  document_number TEXT,                   -- 全建統一様式番号（5, 9 等）
  version INTEGER DEFAULT 1,
  generated_data JSONB NOT NULL,          -- 生成された書類データ
  file_url TEXT,                          -- 生成されたExcel/PDFのURL
  status TEXT DEFAULT 'draft',            -- draft / approved / submitted
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- RLS全テーブル
ALTER TABLE workers ENABLE ROW LEVEL SECURITY;
ALTER TABLE worker_qualifications ENABLE ROW LEVEL SECURITY;
ALTER TABLE construction_sites ENABLE ROW LEVEL SECURITY;
ALTER TABLE site_worker_assignments ENABLE ROW LEVEL SECURITY;
ALTER TABLE safety_documents ENABLE ROW LEVEL SECURITY;
```

2.4 安全書類生成パイプライン

```
# workers/bpo/construction/safety_docs.py

class SafetyDocumentGenerator:
    """
    安全書類自動生成エンジン

    マスタデータ（作業員・資格・現場）から
    全建統一様式の安全書類を自動生成する
    """

    async def generate_worker_roster(
        self,
        site_id: str,
        as_of_date: date = None,
    ) -> SafetyDocument:
        """
        第5号: 作業員名簿を自動生成

        1. site_worker_assignments から対象作業員を取得
        2. workers マスタから個人情報を取得
        3. worker_qualifications から資格情報を取得
        4. 全建統一様式 第5号フォーマットに整形
        5. Excel/PDF を生成
        """

    async def generate_qualification_list(
        self,
        site_id: str,
    ) -> SafetyDocument:
        """第9号: 有資格者一覧表"""

    async def generate_subcontractor_chart(
        self,
        site_id: str,
    ) -> SafetyDocument:
        """第1号-甲: 下請負業者編成表"""

    async def generate_safety_plan(
        self,
        site_id: str,
    ) -> SafetyDocument:
        """
        第6号: 工事安全衛生計画書
        AI生成: 工事内容から危険要因を分析し、安全対策を自動提案
        """

    async def check_expiring_qualifications(
        self,
        company_id: str,
```

```
        days_ahead: int = 90,
    ) -> list[ExpiringQualification]:
        """
        資格有効期限アラート
        期限が近い資格を一覧表示
        """

    async def export_to_greensite_csv(
        self,
        site_id: str,
    ) -> bytes:
        """
        グリーンサイト取り込み用CSVを生成
        （グリーンサイトを使っている元請向け）
        """
```

2.5 UIフロー

```
■ 作業員マスタ管理
  /bpo/workers
  - 作業員一覧（検索・フィルタ）
  - 新規登録 / CSV一括取り込み
  - 資格管理（有効期限アラート付き）
  - 健康診断記録

■ 現場管理
  /bpo/sites
  - 現場一覧（稼働中 / 完了）
  - 作業員アサイン管理（入退場）
  - 安全書類一括生成ボタン

■ 安全書類生成
  /bpo/sites/{id}/safety-docs
  - 書類種別選択 → ワンクリック生成
  - プレビュー → 編集 → 承認
  - Excel/PDF ダウンロード
  - グリーンサイトCSV出力
```

---

3. 出来高・請求書自動生成

3.1 データモデル

```
-- 工事台帳（contract + 出来高管理の基本単位）
CREATE TABLE construction_contracts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  site_id UUID REFERENCES construction_sites(id),
  contract_number TEXT,                    -- 注文番号
  client_name TEXT NOT NULL,               -- 発注者/元請名
  project_name TEXT NOT NULL,              -- 工事名
  contract_amount BIGINT NOT NULL,         -- 契約金額（税抜）
  tax_rate DECIMAL(4,3) DEFAULT 0.10,     -- 消費税率
  contract_date DATE,                      -- 契約日
  start_date DATE,                         -- 着工日
  completion_date DATE,                    -- 竣工予定日
  payment_terms TEXT,                      -- 支払条件（月末締翌月末払い等）
  billing_type TEXT DEFAULT 'monthly',     -- monthly / milestone / completion
  items JSONB NOT NULL,                    -- 内訳 [{name, amount, unit}]
  status TEXT DEFAULT 'active',            -- active / completed / cancelled
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 月次出来高
CREATE TABLE progress_records (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  contract_id UUID NOT NULL REFERENCES construction_contracts(id),
  company_id UUID NOT NULL REFERENCES companies(id),
  period_year INTEGER NOT NULL,            -- 年
  period_month INTEGER NOT NULL,           -- 月
  items JSONB NOT NULL,                    -- [{item_name, contract_amount,
progress_rate, progress_amount}]
  cumulative_amount BIGINT NOT NULL,       -- 累計出来高
  previous_cumulative BIGINT NOT NULL,     -- 前月までの累計
  current_amount BIGINT GENERATED ALWAYS AS (cumulative_amount - previous_cumulative)
STORED,
  status TEXT DEFAULT 'draft',             -- draft / confirmed / billed
  approved_by UUID REFERENCES users(id),
  approved_at TIMESTAMPTZ,
  notes TEXT,
  created_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE(contract_id, period_year, period_month)
);

-- 請求書
CREATE TABLE invoices (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  company_id UUID NOT NULL REFERENCES companies(id),
  contract_id UUID REFERENCES construction_contracts(id),
  progress_record_id UUID REFERENCES progress_records(id),
  invoice_number TEXT NOT NULL,             -- 請求書番号（自動採番）
  invoice_date DATE NOT NULL,               -- 請求日
  due_date DATE NOT NULL,                   -- 支払期日
  client_name TEXT NOT NULL,
  subtotal BIGINT NOT NULL,                 -- 税抜金額
  tax_amount BIGINT NOT NULL,               -- 消費税額
  total BIGINT NOT NULL,                    -- 税込金額
  items JSONB NOT NULL,                     -- 明細
  status TEXT DEFAULT 'draft',              -- draft / sent / paid / overdue
  file_url TEXT,                            -- 生成されたPDF URL
  sent_at TIMESTAMPTZ,
  paid_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- RLS
ALTER TABLE construction_contracts ENABLE ROW LEVEL SECURITY;
ALTER TABLE progress_records ENABLE ROW LEVEL SECURITY;
ALTER TABLE invoices ENABLE ROW LEVEL SECURITY;
```

3.2 出来高・請求パイプライン

```
# workers/bpo/construction/billing.py

class BillingEngine:
    """
    出来高管理 + 請求書自動生成
    """

    async def calculate_progress(
        self,
        contract_id: str,
        period_year: int,
        period_month: int,
        progress_data: list[ProgressInput],
    ) -> ProgressRecord:
        """
        出来高を計算
        - 各内訳項目の進捗率を入力 → 金額を自動計算
        - 前月との差分 = 当月請求額
        - 異常検知: 前月比で大幅な変動があればアラート
        """

    async def generate_invoice(
        self,
        progress_record_id: str,
        format: str = "pdf",
    ) -> Invoice:
        """
        請求書を自動生成
        - 出来高データから請求書を作成
        - 電子帳簿保存法対応（タイムスタンプ不要のクラウド保存）
        - 請求書番号の自動採番
        - PDF/Excel 生成
        """

    async def track_payments(
        self,
        company_id: str,
    ) -> PaymentSummary:
        """
        入金管理
        - 未入金一覧
        - 支払期日超過アラート
        - 資金繰り予測
        """
```

4. 原価管理レポート

4.1 データモデル

```
-- 工事原価（実績）
CREATE TABLE cost_records (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  contract_id UUID NOT NULL REFERENCES construction_contracts(id),
  record_date DATE NOT NULL,
  cost_type TEXT NOT NULL,                 -- material / labor / subcontract / equipment
/ overhead
  description TEXT NOT NULL,
  amount BIGINT NOT NULL,
  vendor_name TEXT,                        -- 仕入先/下請名
  invoice_ref TEXT,                        -- 請求書番号参照
  created_at TIMESTAMPTZ DEFAULT now()
);

ALTER TABLE cost_records ENABLE ROW LEVEL SECURITY;
```

4.2 原価管理エンジン

```
# workers/bpo/construction/cost_report.py

class CostReportEngine:
    """
    原価管理レポート自動生成
    """

    async def generate_monthly_report(
        self,
        contract_id: str,
        year: int,
        month: int,
    ) -> CostReport:
        """
        月次原価レポート
        - 予算（実行予算）vs 実績の差異分析
        - 科目別（材料/労務/外注/経費）の消化率
        - 完成予測原価の推定
        - 赤字リスクのアラート
        """

    async def predict_final_cost(
        self,
        contract_id: str,
    ) -> FinalCostPrediction:
        """
        完成時原価予測（AI）
        - 現在の消化ペースから完成時原価を推定
        - 「このペースだと最終利益率は X%（当初計画: Y%）」
        - 赤字転落リスクがある場合はプロアクティブ提案と連動
        """

    async def generate_company_summary(
        self,
        company_id: str,
        year: int,
        month: int,
    ) -> CompanyCostSummary:
        """
        全社原価サマリー
        - 工事別利益率ランキング
        - 赤字工事一覧
        - 原価率トレンド（3ヶ月移動平均）
        """
```

5. 施工計画書AI生成

```
# workers/bpo/construction/plan_writer.py

class ConstructionPlanWriter:
    """
    施工計画書をAIで生成

    入力: 工事概要（工事名、種別、規模、条件）
    出力: 施工計画書（Word/PDF）

    テンプレートベース + LLMで工事固有の内容を補完
    """

    PLAN_SECTIONS = [
        "工事概要",
        "施工方針",
        "施工体制",
        "主要工種の施工方法",       # ← LLMが工種に応じて生成
        "品質管理計画",
        "安全管理計画",              # ← LLMが危険要因を分析して生成
        "環境対策",
        "工程表",
        "仮設計画",
        "緊急時の連絡体制",
    ]

    async def generate(
        self,
        site_id: str,
        project_type: str,
        work_details: str,        # 工事内容の自由記述 or 仕様書テキスト
    ) -> ConstructionPlan:
        """
        LLMプロンプト:
         「以下の建設工事の施工計画書を作成してください。
          工事内容: {work_details}
          工事種別: {project_type}

          各セクションを具体的に記述してください。
          安全管理計画では、この工種特有の危険要因と対策を明記してください。」
        """
```

6. 工事写真整理

```
# workers/bpo/construction/photo_organizer.py

class PhotoOrganizer:
    """
    工事写真のAI整理 + 電子納品

    入力: 大量の工事写真（JPG/PNG）
    処理:
      1. Document AI で黒板をOCR（工事名、工種、日付、測定値）
      2. LLM/Vision で写真内容を分類（施工前/中/後、工種別）
      3. フォルダ構成を自動生成（工種_日付）
      4. EXIF + 黒板情報でメタデータ付与
    出力: 整理された写真フォルダ + 写真管理台帳
    """

    async def organize(
        self,
        site_id: str,
        photos: list[UploadFile],
    ) -> PhotoOrganizationResult:
        """写真を自動分類・整理"""

    async def generate_photo_ledger(
        self,
        site_id: str,
    ) -> bytes:
        """工事写真台帳（Excel）を生成"""

    async def prepare_electronic_delivery(
        self,
        site_id: str,
        delivery_format: str = "nexco",  # nexco / mlit / prefecture
    ) -> bytes:
        """電子納品フォーマットに変換"""
```

7. 下請管理

```
# workers/bpo/construction/subcontractor.py

class SubcontractorManager:
    """
    下請業者の管理
    - 業者マスタ（評価・実績・資格・保険）
    - 発注管理（注文書発行）
    - 出来高・支払管理
    - 年次評価
    """

    # データは construction_sites + workers + cost_records で管理
    # 追加テーブルは subcontractors（業者マスタ）のみ
```

```
-- 下請業者マスタ
CREATE TABLE subcontractors (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  name TEXT NOT NULL,                       -- 業者名
  representative TEXT,                      -- 代表者名
  address TEXT,
  phone TEXT,
  license_number TEXT,                      -- 建設業許可番号
  license_expiry DATE,                      -- 許可有効期限
  specialties TEXT[],                       -- 得意工種
  insurance JSONB DEFAULT '{}',             -- 保険情報
  evaluation JSONB DEFAULT '{}',            -- 評価 {quality, schedule, safety, price,
overall}
  evaluation_date DATE,
  bank_info JSONB DEFAULT '{}',             -- 振込先
  status TEXT DEFAULT 'active',
  notes TEXT,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

ALTER TABLE subcontractors ENABLE ROW LEVEL SECURITY;
```

8. 許可更新・経審サポート

```
# workers/bpo/construction/license_support.py

class LicenseSupport:
    """
    建設業許可更新 + 経営事項審査（経審）のサポート

    - 許可有効期限の管理（6ヶ月前アラート）
    - 更新必要書類チェックリスト生成
    - 経審申請書の下書き生成
    - 技術者配置のシミュレーション
    """

    async def check_license_status(
        self,
        company_id: str,
    ) -> LicenseStatus:
        """
        許可状況チェック
        - 有効期限まで何ヶ月か
        - 更新に必要なアクション
        - 経管・専技の資格要件充足状況
        """

    async def generate_renewal_checklist(
        self,
        company_id: str,
    ) -> list[ChecklistItem]:
        """
        更新必要書類チェックリスト
        - 財務諸表（直近5年分）
        - 工事経歴書
        - 技術者名簿
        - 社会保険加入証明
        - etc.
        """
```

9. ディレクトリ構成

```
shachotwo-app/
├── workers/
│   └── bpo/
│       ├── __init__.py
│       ├── engine.py                    # BPO共通実行エンジン
│       ├── construction/
│       │   ├── __init__.py
│       │   ├── estimator.py             # ① 積算AI
│       │   ├── safety_docs.py           # ② 安全書類生成
│       │   ├── billing.py               # ③ 出来高・請求書
│       │   ├── cost_report.py           # ④ 原価管理レポート
│       │   ├── plan_writer.py           # ⑤ 施工計画書
│       │   ├── photo_organizer.py       # ⑥ 工事写真整理
│       │   ├── subcontractor.py         # ⑦ 下請管理
│       │   ├── license_support.py       # ⑧ 許可更新・経審
│       │   └── models.py               # Pydanticモデル（共通）
│       └── common/
│           ├── __init__.py
│           ├── document_gen.py          # Excel/PDF/Word生成
│           └── template_engine.py       # テンプレートエンジン
│
├── routers/
│   ├── estimation.py                    # 積算API
│   ├── safety_docs.py                   # 安全書類API
│   ├── billing.py                       # 出来高・請求API
│   ├── cost_management.py               # 原価管理API
│   ├── construction_sites.py            # 現場管理API
│   ├── workers_management.py            # 作業員管理API
│   └── subcontractors.py               # 下請管理API
│
├── db/
│   └── migrations/
│       ├── 006_construction_bpo.sql     # 積算・現場・作業員・出来高テーブル
│       └── 007_safety_documents.sql     # 安全書類テーブル
│
├── frontend/src/app/(authenticated)/
│   └── bpo/
│       ├── layout.tsx                   # BPOセクション共通レイアウト
│       ├── page.tsx                     # BPOダッシュボード
│       ├── estimation/
│       │   ├── page.tsx                 # 積算プロジェクト一覧
│       │   ├── new/page.tsx             # 新規積算（ステップフォーム）
│       │   ├── [id]/page.tsx            # 積算詳細・編集
│       │   └── prices/page.tsx          # 単価マスタ管理
│       ├── sites/
│       │   ├── page.tsx                 # 現場一覧
│       │   ├── new/page.tsx             # 現場登録
│       │   ├── [id]/page.tsx            # 現場詳細
```

```
│       │   ├── [id]/workers/page.tsx    # 作業員アサイン
│       │   └── [id]/safety/page.tsx     # 安全書類生成
│       ├── billing/
│       │   ├── page.tsx                # 請求一覧
│       │   ├── contracts/page.tsx      # 工事台帳
│       │   └── [id]/page.tsx           # 出来高・請求書作成
│       ├── costs/
│       │   └── page.tsx                # 原価管理ダッシュボード
│       ├── workers/
│       │   ├── page.tsx                # 作業員マスタ
│       │   └── [id]/page.tsx           # 作業員詳細・資格管理
│       └── subcontractors/
│           ├── page.tsx                # 下請業者一覧
│           └── [id]/page.tsx           # 業者詳細・評価
```

---

10. API設計

積算

```
POST   /api/estimation/projects              # プロジェクト作成
GET    /api/estimation/projects              # 一覧取得
GET    /api/estimation/projects/{id}         # 詳細取得
PATCH  /api/estimation/projects/{id}         # 更新
DELETE /api/estimation/projects/{id}         # 削除

POST   /api/estimation/projects/{id}/documents   # 図面・設計書アップロード
POST   /api/estimation/projects/{id}/extract     # 数量抽出（AI）
POST   /api/estimation/projects/{id}/suggest-prices  # 単価推定（AI）
POST   /api/estimation/projects/{id}/calculate   # 諸経費計算
POST   /api/estimation/projects/{id}/export      # 内訳書出力（Excel/PDF）

GET    /api/estimation/prices                # 単価マスター一覧
POST   /api/estimation/prices/import         # 過去積算書一括取り込み
GET    /api/estimation/labor-rates           # 公共工事設計労務単価
GET    /api/estimation/templates             # 積算テンプレート一覧
```

現場・作業員・安全書類

```
POST   /api/sites                          # 現場登録
GET    /api/sites                          # 現場一覧
GET    /api/sites/{id}                     # 現場詳細
PATCH  /api/sites/{id}                     # 現場更新

POST   /api/sites/{id}/workers             # 作業員アサイン
DELETE /api/sites/{id}/workers/{worker_id}   # 作業員退場

POST   /api/sites/{id}/safety-docs/generate  # 安全書類一括生成
GET    /api/sites/{id}/safety-docs          # 安全書類一覧
GET    /api/sites/{id}/safety-docs/{doc_id}/download  # ダウンロード
POST   /api/sites/{id}/safety-docs/greensite-export   # グリーンサイトCSV出力

POST   /api/workers                        # 作業員登録
GET    /api/workers                        # 作業員一覧
GET    /api/workers/{id}                   # 作業員詳細
PATCH  /api/workers/{id}                   # 作業員更新
POST   /api/workers/import                 # CSV一括取り込み
GET    /api/workers/expiring-qualifications   # 期限切れ間近の資格
```

出来高・請求・原価

```
POST   /api/contracts                      # 工事台帳登録
GET    /api/contracts                      # 工事台帳一覧
GET    /api/contracts/{id}                 # 工事台帳詳細

POST   /api/contracts/{id}/progress        # 出来高登録
GET    /api/contracts/{id}/progress        # 出来高履歴
POST   /api/contracts/{id}/progress/{pid}/invoice  # 請求書生成

GET    /api/invoices                         # 請求書一覧
GET    /api/invoices/{id}/download           # 請求書ダウンロード
PATCH  /api/invoices/{id}/status             # ステータス更新（sent/paid）

GET    /api/costs/report                   # 原価レポート
GET    /api/costs/prediction/{contract_id}  # 完成時原価予測
```

---

11. 並列開発プラン

Phase A: Week 1-2（6エージェント並列）

```
Agent 1: DB — migrations/006_construction_bpo.sql
         全テーブル作成 + RLS + インデックス

Agent 2: workers/bpo/construction/estimator.py + models.py
         積算AIパイプライン（拾い出し + 単価推定 + 諸経費計算 + 内訳書生成）

Agent 3: workers/bpo/construction/safety_docs.py
         安全書類自動生成（作業員名簿、有資格者一覧、施工体制台帳）

Agent 4: workers/bpo/common/document_gen.py
         Excel/PDF/Word生成エンジン（openpyxl + reportlab + python-docx）

Agent 5: routers/estimation.py + routers/safety_docs.py + routers/construction_sites.py
+ routers/workers_management.py
         FastAPIルーター全部

Agent 6: frontend/src/app/(authenticated)/bpo/
         BPOダッシュボード + 積算画面 + 現場管理画面 + 安全書類画面
```

Phase B: Week 3（4エージェント並列）

```
Agent 1: workers/bpo/construction/billing.py
         出来高・請求書エンジン

Agent 2: workers/bpo/construction/cost_report.py
         原価管理レポート + 赤字予測AI

Agent 3: routers/billing.py + routers/cost_management.py
         ルーター

Agent 4: frontend 出来高・請求・原価画面
```

Phase C: Week 4（3エージェント並列）

```
Agent 1: workers/bpo/construction/plan_writer.py
         施工計画書AI生成

Agent 2: workers/bpo/construction/photo_organizer.py
         工事写真整理（Document AI + Vision）

Agent 3: frontend 施工計画書・写真整理画面
```

Phase D: Week 5（3エージェント並列）

```
Agent 1: workers/bpo/construction/subcontractor.py + license_support.py
         下請管理 + 許可更新

Agent 2: routers/subcontractors.py
         ルーター + テスト

Agent 3: frontend 下請管理・許可更新画面 + BPO全体の仕上げ
```

---

12. 単価データ戦略（重要制約対応）

問題

```
単価データはすべて有償 or PDF:
  - 建設物価調査会のデータファイル: 有償ライセンス（推定年間50-200万円）
  - 経済調査会のデータベース: 有償ライセンス
  - 公共工事設計労務単価: 無償だがPDFのみ（APIなし）
  - 各積算ソフトの内蔵単価: ソフトライセンスに紐づく

→ MVP段階で年間50-200万のデータライセンス費は避けたい
```

戦略: 「育つ単価DB」アプローチ

```
■ Day 1（ゼロデータ状態）
  ユーザーができること:
    ① 数量計算書Excelをアップロード → 数量を自動構造化
    ② 単価は手動入力（自社の見積単価・実績単価）
    ③ LLMが「一般的な単価レンジ」を参考情報として表示

  → 最初の1件は手間がかかるが、内訳書自動生成の価値はある

■ Week 1（初期データ蓄積）
  ④ 公共工事設計労務単価をDB化（無償公開データ）
     - 国交省PDF → OCR/手動 → public_labor_rates テーブル
     - 51職種 × 47都道府県 × 3年分
  ⑤ ユーザーが過去の積算書Excelを一括取り込み
     - 「過去1年分の積算書をアップロードしてください」
     - → 自動で工種×単価の実績DBが構築される

■ Month 1（実用レベル）
  ⑥ 自社実績から「推奨単価」を自動候補表示
     - 同一工種・同一地域の過去実績平均
     - 信頼度付き（実績件数が多いほど信頼度UP）
  ⑦ 時期補正（年度更新対応）
     - 労務単価の年度変動率を適用

■ Month 3+（ネットワーク効果）
  ⑧ 匿名化された他社実績からの市場単価推定
     - 同一地域・同一工種のN社データ（匿名化）
     - 「同業他社の直近3ヶ月平均: ¥XX,XXX（N=5社）」

  ⑨ 建設物価データライセンス契約の検討
     - 顧客50社超で採算ライン
     - データライセンス費を月額に転嫁

■ 代替データソース（追加調査が必要）
  ⑩ 国土交通省「公共工事の落札情報」
     - 落札金額は公開情報 → 単価逆算の可能性
  ⑪ 各自治体の設計単価公表資料
     - 一部自治体はExcelで単価表を公開
  ⑫ JACIC（日本建設情報総合センター）の公開データ
```

---

13. 既存アーキテクチャとの統合

Brain × BPO の連携

```
┌──────────────────────┐      ┌──────────────────────────────┐
│ Brain（既存）         │      │ BPO（新規）                    │
│                      │      │                               │
│ knowledge/           │──────→│ 積算: ナレッジから見積基準参照    │
│  - 見積ルール        │      │ 安全: ナレッジから安全基準参照    │
│  - 承認フロー        │      │ 請求: ナレッジから支払条件参照    │
│                      │      │                               │
│ proactive/           │←─────│ 赤字予測 → 能動提案              │
│  - リスクアラート    │      │ 資格期限 → リスクアラート        │
│  - 改善提案          │      │ 許可更新 → リマインダー          │
│                      │      │                               │
│ genome/              │──────→│ 建設業テンプレートの拡張          │
│  - construction.json│      │ 積算テンプレート追加             │
└──────────────────────┘      └──────────────────────────────┘
```

main.py へのルーター追加

```
# 建設BPO ルーター
from routers.estimation import router as estimation_router
from routers.safety_docs import router as safety_docs_router
from routers.construction_sites import router as sites_router
from routers.workers_management import router as workers_router
from routers.billing import router as billing_router
from routers.cost_management import router as cost_router
from routers.subcontractors import router as subcontractors_router

app.include_router(estimation_router, prefix="/api/estimation", tags=["estimation"])
app.include_router(safety_docs_router, prefix="/api/safety-docs", tags=["safety-docs"])
app.include_router(sites_router, prefix="/api/sites", tags=["sites"])
app.include_router(workers_router, prefix="/api/workers", tags=["workers"])
app.include_router(billing_router, prefix="/api/billing", tags=["billing"])
app.include_router(cost_router, prefix="/api/costs", tags=["costs"])
app.include_router(subcontractors_router, prefix="/api/subcontractors", tags=
["subcontractors"])
```

フロントエンド ナビゲーション拡張

```
既存:
  ダッシュボード | ナレッジ入力 | ナレッジ一覧 | Q&A | 提案

追加:
  BPO ▼
    ├── ダッシュボード（BPO全体の状況）
    ├── 積算
    ├── 現場管理
    ├── 安全書類
    ├── 出来高・請求
    ├── 原価管理
    ├── 作業員管理
    └── 下請管理
```

---

14. 依存ライブラリ追加

```
# requirements.txt に追加
openpyxl>=3.1.0          # Excel読み書き
reportlab>=4.0           # PDF生成
python-docx>=1.0         # Word生成
Pillow>=10.0             # 画像処理
```

---

15. リスクと対策

リスク	影響	対策
単価データがない	積算AIの精度が低い	「育つDB」戦略 + 過去積算書一括取込で初期データ確保
図面OCRの精度	数量拾い出しがうまくいかない	数量計算書Excel取込を優先。図面OCRは補助的に
安全書類のフォーマット差異	元請ごとにフォーマットが違う	全建統一様式を基本 + カスタムテンプレート対応
グリーンサイトとの競合	「うちはグリーンサイト使ってる」と言われる	競合ではなく補完（CSVエクスポート機能）
電子帳簿保存法対応	請求書の法的要件	クラウド保存+改ざん防止ログで対応
ANDPAD APIの制限	連携できる範囲が限られる	CSV取込をフォールバックとして常に用意

16. KPI・成功基準

Phase A 完了基準（Week 2末）

- [ ] 数量計算書Excel → 構造化抽出が動作する
- [ ] 単価手動入力 → 内訳書Excel出力が動作する
- [ ] 作業員マスタ登録 → 作業員名簿（第5号）PDF出力が動作する
- [ ] パイロット1社で「使ってみたい」と言われる

全体完了基準（Week 5末）

- [ ] 8モジュール全て稼働
- [ ] パイロット3社が実業務で使用開始
- [ ] 月間工数削減: 1社あたり40時間以上
- [ ] 積算: 1案件の積算時間が50%以上短縮

PMF指標

- [ ] NPS ≥ 30
- [ ] 「なくなったら困る」≥ 60%
- [ ] 月額30万円で「安い」と感じる社数 ≥ 50%

---

17. 図面自動拾い出しAI — 要件定義（Phase 2）

> **位置づけ**: Section 1.2 の Level 1「拾い出しAI」の拡張。
> 現在の Level 1 は「数量計算書Excel/テキスト → 構造化抽出」だが、
> 本セクションでは**「図面PDF/画像 → 部材認識 → 数量自動算出」**を実現する。
> **目標**: 積算士の拾い出し作業を 1/4〜1/8 に削減（完全自動ではなく AI提案+人間確認）

17.1 なぜ図面自動拾い出しが必要か

```
現在のLevel 1の限界:
  入力 = 数量計算書Excel or テキスト
  → 「数量計算書が既にある」ことが前提
  → しかし中小建設では数量計算書を作ること自体が負担
  → 社長は「図面を渡したら全部やってほしい」

積算BPOの構造的矛盾:
  概算（±20-30%）で出す → 「自分でもできる。金払う意味ない」
  精密（±5%）で出す     → 数量拾い出しが必要 → 図面を読む力が必要
  → 図面自動拾い出しなしではBPOとしての価値が薄い

競合状況:
  ・燈（Akari Inc.）: 企業価値1,000億、東大エンジニア415名、大手ゼネコン向け
  ・Kreo: AI拾い出し精度98.5%、訓練データ1.5M件（海外）
  ・TEMOTO: 面積の自動拾い出し（個数物 精度90%+）
  → 大手向けは燈が支配。中小向けは空白地帯。ここを取る
```

17.2 技術アプローチ — ハイブリッドCV+LLMパイプライン

設計方針: 専用CVモデルの単独ではなく、CV + OCR + マルチモーダルLLM + ルールエンジンの4層で精度を積み上げる。完全自動を目指さず「AI提案 → 人間確認」のワークフローで実質精度98%+を実現する。

```
┌──────────────────────────────────────────────────┐
│                 入力: 図面PDF（複数枚）                 │
│                 平面図 / 断面図 / 構造図 / 仕上げ表      │
└──────────────────────────────────────────────────┘
                        │
                        ▼
┌──────────────────────────────────────────────────┐
│  Layer 1: 前処理（OpenCV）                            │
│  ・PDF → 高解像度画像変換（300dpi+）                    │
│  ・二値化・ノイズ除去・傾き補正                          │
│  ・線分検出（ハフ変換）→ 壁候補線                       │
│  ・矩形検出 → 柱候補                                   │
│  ・図面種別の自動判定（平面図/断面図/構造図）            │
│  ・縮尺の自動検出                                      │
│                                                      │
│  技術: OpenCV, pdf2image, NumPy                       │
│  精度目標: 95%+（確立された技術）                       │
└──────────────────────────────────────────────────┘
                        │
                        ▼
┌──────────────────────────────────────────────────┐
│  Layer 2: 文字・寸法読み取り（OCR）                    │
│  ・Google Document AI で全文字抽出（日本語対応）        │
│  ・寸法パターンの正規表現パーサー                       │
│    例: "600×600", "D25@100", "φ30", "3,500"          │
│  ・通り芯記号の検出（X1, Y1 / イロハニ / 1234）        │
│  ・凡例テーブル・仕上げ表の構造化                       │
│  ・部材符号の検出（C1, G1, W1, S1...）                 │
│                                                      │
│  技術: Google Document AI, Tesseract（予備）, 正規表現 │
│  精度目標: 90%+                                       │
└──────────────────────────────────────────────────┘
                        │
                        ▼
┌──────────────────────────────────────────────────┐
│  Layer 3: 部材検出（物体検出モデル）                   │
│  ・RT-DETR or YOLO-NAS で壁・柱・開口部・階段等を検出 │
│  ・セマンティックセグメンテーション（壁領域分割）       │
│  ・転移学習: BIM生成データ + パイロット企業図面         │
│  ・マルチモーダルLLM併用:                              │
│    CVが低確信度の部材 → Gemini Visionに判定委託        │
│                                                      │
│  ★ ライセンス判断:                                     │
│    YOLOv8 = AGPL-3.0（SaaS提供時にソースコード公開義務）│
│    RT-DETR / YOLO-NAS = Apache 2.0（公開義務なし）      │
│    → Apache 2.0モデルを採用。LICENSE表示のみで商用自由 │
│                                                      │
│  検出クラス（Phase 2）:                                │
│    壁(耐力壁/間仕切り), 柱, 梁, ドア, 窓,              │
│    階段, 通り芯, 寸法線, 建具記号                      │
│  追加クラス（Phase 3）:                                │
│    配筋記号, 断面詳細, 仕口, 設備記号                  │
│                                                      │
│  技術: RT-DETR / YOLO-NAS (Apache 2.0), SegNeXt,      │
```

```
│        Gemini Vision                                    │
│  精度目標: 85-90% (mAP@0.5)                              │
└──────────────────────────────────────────────────────────┘
                    ┃
                    ▼

┌──────────────────────────────────────────────────────────┐
│  Layer 4: 意味理解・複数図面統合 (LLM + Graph)            │
│  ・検出部材への意味付与                                    │
│    矩形(600×600) @ (X1,Y1) → 「RC柱 C1」                 │
│    太い線(200mm厚) → 「RC耐力壁」                         │
│  ・通り芯ベースの座標系構築                                │
│  ・BuildingGraph: 複数図面のエンティティマッチング        │
│    平面図のC1 = 構造図のC1 = 断面図の(X1,Y1)柱            │
│  ・マッチングキー: (部材種別, 通り芯位置, 階)             │
│  ・ファジーマッチング: 通り芯表記の揺れ対応               │
│                                                          │
│  技術: Gemini/Claude LLM, NetworkX (グラフ構造)           │
│  精度目標: 80-90% (マッチング精度)                        │
└──────────────────────────────────────────────────────────┘
                    ┃
                    ▼

┌──────────────────────────────────────────────────────────┐
│  Layer 5: 数量算出 (ルールエンジン)                       │
│  ・統合BuildingGraphから部材タイプ別に数量計算            │
│    コンクリート = 断面積 × 高さ (m³)                      │
│    型枠 = 周長 × 高さ (m²)                                │
│    鉄筋 = 本数 × 径 × 長さ × 単位重量 (t)                 │
│    仕上げ = 部屋面積 × 仕上げ種別 (m²)                    │
│  ・積算ルールDB適用 (ロス率・割増率・端数処理)            │
│  ・既存estimator.pyの単価マッピング+内訳書生成と接続      │
│                                                          │
│  技術: Python (NumPy/Pandas), 積算ルールDB (JSON)         │
└──────────────────────────────────────────────────────────┘
                    ┃
                    ▼

┌──────────────────────────────────────────────────────────┐
│  Layer 6: 人間確認ループ (確認UI)                         │
│  ・図面ビューア: 検出結果を図面上にオーバーレイ表示       │
│  ・確信度カラー表示 (緑=高確信 / 黄=中 / 赤=要確認)       │
│  ・部材クリック → 詳細表示・寸法修正・種別変更            │
│  ・修正データ → Supabaseに蓄積 → 月次再訓練に利用         │
│  ・確認完了 → Layer 5の数量算出を再実行 → 内訳書出力      │
│                                                          │
│  技術: Next.js + Canvas API + shadcn/ui                  │
│  目標: AI 90% + 人間確認 = 98%+の実質精度                 │
└──────────────────────────────────────────────────────────┘
```

17.3 データモデル（追加テーブル）

```
-- 図面ファイル管理
CREATE TABLE drawing_files (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  project_id UUID NOT NULL REFERENCES estimation_projects(id),
  file_name TEXT NOT NULL,
  file_path TEXT NOT NULL,               -- Supabase Storage パス
  drawing_type TEXT NOT NULL,            -- floor_plan / cross_section / structural /
finish_schedule / foundation
  floor TEXT,                            -- 1F, 2F, RF等
  scale TEXT,                            -- "1:100", "1:50"等
  ocr_text JSONB DEFAULT '{}',          -- Document AI OCR結果
  grid_system JSONB DEFAULT '{}',       -- {"X": ["X1","X2"...], "Y": ["Y1","Y2"...]}
  processing_status TEXT DEFAULT 'pending', -- pending / processing / completed / failed
  created_at TIMESTAMPTZ DEFAULT now()
);

-- 検出部材
CREATE TABLE detected_elements (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  drawing_file_id UUID NOT NULL REFERENCES drawing_files(id),
  project_id UUID NOT NULL REFERENCES estimation_projects(id),
  element_type TEXT NOT NULL,            -- column / beam / wall / door / window / slab
/ stairs / rebar
  element_id TEXT,                       -- C1, G1, W1等（図面上の部材符号）
  grid_position TEXT,                    -- "(X1,Y1)" 通り芯位置
  floor TEXT,                            -- 階
  bbox JSONB NOT NULL,                   -- {"x": 100, "y": 200, "w": 50, "h": 50} 図面上
の位置
  dimensions JSONB DEFAULT '{}',        -- {"width": 600, "depth": 600, "height": 3500}
mm単位
  properties JSONB DEFAULT '{}',        -- 部材固有属性 {"rebar": "12-D25", "hoop":
"D13@100"} 等
  confidence FLOAT NOT NULL,            -- AI確信度 0.0-1.0
  detection_source TEXT NOT NULL,       -- yolo / llm_vision / ocr / manual
  human_verified BOOLEAN DEFAULT false, -- 人間確認済みか
  human_correction JSONB,              -- 人間による修正内容（再学習データに使用）
  created_at TIMESTAMPTZ DEFAULT now()
);

-- BuildingGraph: 複数図面の部材統合
CREATE TABLE building_elements (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  project_id UUID NOT NULL REFERENCES estimation_projects(id),
  element_type TEXT NOT NULL,
  element_id TEXT NOT NULL,              -- 統合後の部材ID（C1等）
  grid_position TEXT,
```

```
  floors TEXT[] DEFAULT '{}',           -- {"1F", "2F", "3F"} 存在する階のリスト
  merged_dimensions JSONB DEFAULT '{}', -- 複数図面から統合した寸法情報
  merged_properties JSONB DEFAULT '{}', -- 複数図面から統合した属性情報
  source_elements UUID[] DEFAULT '{}', -- detected_elements.id のリスト（統合元）
  match_confidence FLOAT,              -- マッチング確信度
  quantities JSONB DEFAULT '{}',       -- 算出済み数量 {"concrete_m3": X, "formwork_m2": Y,
"rebar_ton": Z}
  created_at TIMESTAMPTZ DEFAULT now()
);

-- 訓練データ管理（フィードバックループ用）
CREATE TABLE training_samples (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  drawing_file_id UUID NOT NULL REFERENCES drawing_files(id),
  image_path TEXT NOT NULL,             -- 訓練用画像のStorageパス
  annotations JSONB NOT NULL,           -- YOLOフォーマットのアノテーション
  source TEXT NOT NULL,                 -- bim_generated / pilot_company /
user_correction / cubicasa
  quality_score FLOAT,                 -- アノテーション品質スコア
  used_in_training BOOLEAN DEFAULT false,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- RLS: 全テーブルにcompany_idベースのRLS適用
ALTER TABLE drawing_files ENABLE ROW LEVEL SECURITY;
ALTER TABLE detected_elements ENABLE ROW LEVEL SECURITY;
ALTER TABLE building_elements ENABLE ROW LEVEL SECURITY;
ALTER TABLE training_samples ENABLE ROW LEVEL SECURITY;
```

17.4 訓練データ戦略

データソース

ソース	入手方法	量	コスト	用途
CubiCasa5k	GitHub公開 (github.com/CubiCasa/CubiCasa5k)	5,000枚 + SVGアノテーション	無料	⚠️ **実験・研究用のみ（CC BY-NC-SA 4.0 = 商用NG）。本番モデルには含めない**
Roboflow Floor Plans	Roboflow Universe	500枚 + アノテーション	無料	学習パイプライン検証用
国交省BIM → 2D図面自動生成	営繕BIMモデル（Revit/IFC公開）	**無限生成可能**（3モデル × Nバリエーション）	無料	**正解付き日本語図面データ（最重要）**
BIM審査サンプル	BLCJ公開（S造事務所 / RC造庁舎）	2モデル	無料	正解付き日本語図面データ
パイロット企業の実図面	営業・税理士経由で収集	100-300枚	営業コスト	実データの多様性確保
確認UIの修正データ（運用後）	本番運用からの自動蓄積	月500枚（10社×月10案件×5枚）	自動	継続的精度改善（フライホイール）

CubiCasa5k / Roboflowの限界と対策

```
★重要: CubiCasa5k・Roboflowは欧米の図面データ

学習できる（転移可能）:
  ✅ 壁の線分パターン（2本の平行線 = 壁）
  ✅ 開口部の表現パターン（壁の途切れ = ドア/窓）
  ✅ 部屋の領域分割の概念
  ✅ 図面の基本的な幾何構造

学習できない（日本固有、追加学習が必要）:
  ❌ 日本語テキスト
  ❌ JIS建具記号（引き違い戸、片開き戸、折戸等）
  ❌ 和室の畳表現
  ❌ 日本特有の間取り（玄関、土間、押入れ）
  ❌ RC構造図の配筋表記（D25@100）
  ❌ 通り芯の日本式表記

対策: CubiCasaは実験用のみ（CC BY-NC-SA 4.0 = 商用利用NG）
  本番モデルの訓練データ:
  ① 国交省BIM → 2D図面自動生成（公開データ、商用OK）
  ② パイロット企業の実図面（許諾取得）
  ③ 確認UIの修正データ（自社蓄積）
  → CubiCasa5kは一切含めない → ライセンス問題なし
```

BIM → 2D図面 自動生成パイプライン（訓練データ量産の仕組み）

```
入力: 国交省公開BIMモデル（Revit/IFC形式）
  │
  ├─ Revit API or IfcOpenShell で自動処理
  │
  ▼
2D図面を自動生成（複数パターン）
  ├─ 平面図（各階）
  ├─ 断面図（各通り）
  ├─ 構造図（柱・梁リスト、配筋詳細）
  └─ 仕上げ表
  │
  ▼
同時にBIMから正解データを自動抽出
  ├─ 全柱リスト（位置・断面・高さ・配筋）
  ├─ 全梁リスト（スパン・断面・配筋）
  ├─ 全壁リスト（長さ・厚さ・高さ・開口）
  ├─ 全開口部リスト（種類・寸法）
  ├─ 正確な数量（コンクリートm³・型枠m²・鉄筋t）
  └─ YOLOフォーマットのバウンディングボックス
  │
  ▼
データ拡張でバリエーション生成
  ├─ 縮尺変更（1:50, 1:100, 1:200）
  ├─ 線の太さ変更
  ├─ ノイズ追加（FAX品質シミュレート）
  ├─ 解像度変更（150dpi, 200dpi, 300dpi）
  ├─ 傾き追加（スキャン歪みシミュレート）
  └─ 部分的な手書き風変換

結果: BIMモデル1個 → 数十〜数百パターンの正解付き訓練データ
```

段階的な訓練計画

```
Stage 1（Phase 0）: BIM生成データ + 公開データでベースモデル構築
  データ: BIM→2D自動生成 300枚 + Roboflow公開データ
  学習: 壁・開口部・部屋領域の基本概念
  ※ CubiCasa5kは実験で精度検証に使うが、本番モデルには含めない（CC BY-NC-SA）
  精度: 日本図面に適用で60-70%

Stage 2（Phase 1前半）: BIM生成データで日本図面適応
  データ: +300枚（BIM自動生成、正解付き）
  学習: JIS記号・日本語・通り芯・和室
  精度: 80-85%

Stage 3（Phase 1後半）: パイロット企業の実データ追加
  データ: +200枚（実データ、手動アノテーション）
  学習: 手書き図面、FAX品質、設計事務所ごとの描き方の差異
  精度: 85-90%

Stage 4（運用後）: 確認ループからの自動蓄積
  データ: +500枚/月（確認UI修正データ）
  学習: 月次再訓練で継続改善
  精度: 6ヶ月で90-95%（Kreoと同じフライホイール）
```

17.5 アノテーション仕様

検出クラス定義

クラスID	クラス名	日本語	検出方法	Phase
0	`bearing_wall`	耐力壁	YOLO + 線幅判定	2
1	`partition_wall`	間仕切り壁	YOLO + 線幅判定	2
2	`column`	柱	YOLO（矩形検出）	2
3	`beam`	梁	YOLO（構造図から）	3
4	`door`	ドア	YOLO + JIS記号認識	2
5	`window`	窓	YOLO + JIS記号認識	2
6	`stairs`	階段	YOLO	2
7	`slab`	スラブ	セグメンテーション	3
8	`grid_line`	通り芯	OCR + 線分検出	2
9	`dimension`	寸法線	OCR + 線分検出	2
10	`rebar_spec`	配筋仕様	OCR + 正規表現	3
11	`room_name`	部屋名	OCR	2

アノテーションフォーマット（YOLO形式）

```
# 各図面画像に対応する .txt ファイル
# クラスID　中心X　中心Y　幅　高さ　（全て0-1の正規化座標）
2 0.234 0.456 0.032 0.032    # column at (X1,Y1)
0 0.234 0.500 0.005 0.200    # bearing_wall
4 0.300 0.456 0.020 0.010    # door
```

アノテーション作業フロー

```
1. Roboflow にプロジェクト作成（クラス定義をインポート）
2. BIM生成図面は正解から自動アノテーション（手動作業ゼロ）
3. パイロット企業図面は手動アノテーション
    ・1枚あたり約30分（壁・柱・開口部のラベル付け）
    ・積算士が品質レビュー（1枚5分）
4. 確認UIの修正は自動でアノテーション形式に変換
```

17.6 技術スタック詳細

必須ライブラリ・サービス

```
# Layer 1: 前処理
pdf2image==1.17+          # PDF→画像変換（poppler依存）
opencv-python==4.9+       # 画像処理・線分検出・矩形検出
Pillow==10.0+             # 画像操作
numpy==1.26+              # 数値計算

# Layer 2: OCR
google-cloud-documentai   # Google Document AI SDK
pytesseract               # Tesseract OCR（フォールバック）

# Layer 3: 物体検出
# ★ ライセンス: Apache 2.0を選択（ソースコード公開義務なし）
#   YOLOv8 = AGPL-3.0（SaaS提供時にソースコード公開義務 → 不採用）
#   RT-DETR / YOLO-NAS = Apache 2.0（LICENSE表示のみで商用自由 → 採用）
torch==2.2+               # PyTorch
torchvision==0.17+        # 画像変換
# RT-DETR: pip install transformers  (Hugging Face経由)
# YOLO-NAS: pip install super-gradients  (Deci AI提供)

# Layer 4: 意味理解・グラフ
networkx==3.2+            # BuildingGraph構造
google-genai              # Gemini Vision API

# Layer 5: 数量算出
pandas==2.1+              # 集計・テーブル操作
openpyxl==3.1+            # Excel出力（既存estimator.pyと共有）

# Layer 6: 確認UI
# Next.js + Canvas API + shadcn/ui（既存フロントエンド拡張）

# アノテーション・訓練管理
roboflow==1.1+            # Roboflow SDK（データセット管理）
```

インフラ要件

用途	サービス	スペック	月額コスト
モデル訓練	GCP Vertex AI / Compute Engine	NVIDIA A100 or L4	¥1万/月（月1回再訓練、数時間）
推論（本番）	GCP Cloud Run GPU or Compute Engine	NVIDIA T4 or L4	¥5-10万/月
LLM API	Gemini 2.5 Flash	Vision入力対応	¥1-3万/月
OCR API	Google Document AI	日本語対応	¥1-2万/月
ストレージ	Supabase Storage + GCS	図面画像・訓練データ	¥数千/月
合計			**¥10-15万/月（本番運用時）**

17.7 外部リソース要件

リソース	必要量	調達方法	コスト
積算士（業務委託・段階縮小）	Phase1: フル稼働 → Phase3: 月4時間 → 卒業	業務委託	Phase1: ¥30-50万/月 → Phase3: ¥2-5万/月 → ¥0
パイロット企業	3-5社	建設業の知人・税理士経由	無償 or 割引提供
建設物価（単価DB）	書籍1冊	建設物価調査会	¥5万/年
公共工事設計労務単価	PDF	国交省が毎年公開（無料）	無料
国交省BIMモデル	Revit/IFC	営繕BIMモデル公開（無料）	無料
BIM審査サンプル	Revit/IFC	BLCJ公開（無料）	無料
CubiCasa5k	5,000枚	GitHub公開（無料）	無料

17.8 開発ロードマップ

```
Phase 0（1ヶ月）: 前処理 + OCR + ベースモデル
  ├─ Week 1-2: pdf2image + OpenCV前処理パイプライン
  ├─ Week 3-4: Document AI OCR + 寸法パーサー
  └─ Week 3-4: BIM生成データでRT-DETR/YOLO-NAS転移学習（並列）
  成果: 日本図面で壁・ドア・窓を70%+検出

Phase 1（3ヶ月）: 日本図面対応 + 構造図
  ├─ Week 5-6: BIM→2D図面生成パイプライン構築
  ├─ Week 7-8: BIM生成データでYOLOv8追加学習
  ├─ Week 9-10: パイロット企業図面のアノテーション+学習
  ├─ Week 11-12: 構造図対応（配筋記号・断面詳細）
  └─ Week 13-16: マルチモーダルLLM併用テスト
  成果: 日本の平面図+構造図で85-90%検出

Phase 2（3ヶ月）: 複数図面統合 + 数量算出
  ├─ Week 17-18: 通り芯検出モジュール
  ├─ Week 19-22: BuildingGraph構築（エンティティマッチング）
  ├─ Week 23-24: BIMサンプルで統合精度検証
  └─ Week 25-26: 数量算出ルールエンジン
  成果: 複数図面→統合→数量ドラフトの自動生成

Phase 3（2ヶ月）: 確認UI + フィードバックループ
  ├─ Week 27-30: 図面ビューア（Canvas API、オーバーレイ表示）
  ├─ Week 31-32: 確認・修正UI + 修正データ蓄積
  ├─ Week 33-34: 再訓練パイプライン（月次バッチ）
  └─ Week 35-36: 既存estimator.pyとの統合
  成果: 図面アップロード → AI拾い出し → 人間確認 → 内訳書出力

Phase 4（継続）: 精度改善フライホイール
  ├─ 月次: 確認UI修正データで再訓練
  ├─ 四半期: 新しいBIMモデルから訓練データ追加
  └─ 半年後: 精度90-95%到達（設備図対応はPhase 5+）
```

17.9 精度目標と測定方法

指標	Phase 0末	Phase 1末	Phase 2末	Phase 3末	6ヶ月運用後
部材検出 mAP@0.5	80%（洋風）	85-90%（日本）	—	—	90-95%
寸法読み取り精度	—	90%+	—	—	95%+
複数図面マッチング精度	—	—	80-90%	—	90%+
数量算出精度（vs 手動）	—	—	—	±15-20%	±5-10%
実質精度（AI+人間確認）	—	—	—	**95%+**	**98%+**

```
精度測定方法:
  1. BIMモデルから「正解数量」を自動算出
  2. 同じBIMから2D図面を生成
  3. AIパイプラインで図面を解析 → 数量算出
  4. 正解数量 vs AI数量 を比較
  → BIMがあれば自動で精度測定できる（人手不要）
```

17.10 既存システムとの接続

```
図面自動拾い出しAI（本セクション）
  │
  ▼ 数量リスト（detected_elements → building_elements）
  │
  ▼ Layer 5の数量算出結果
  │
  ├──→ estimation_items テーブルに格納（既存）
  │
  ▼ 既存 estimator.py の処理フロー
  │
  ├─ suggest_unit_prices()  ← 既存の単価推定をそのまま使用
  ├─ calculate_overhead()   ← 既存の諸経費計算をそのまま使用
  └─ generate_breakdown_data() ← 既存の内訳書生成をそのまま使用

つまり:
  図面自動拾い出しAI = 「estimator.pyの入力を自動化するモジュール」
  estimator.py自体は変更不要（入力が手動→自動に変わるだけ）
```

17.11 コスト総括

項目	費用	備考
GPU（訓練）	¥10万	初回+月次再訓練10ヶ月分
GPU（推論/本番）	¥50-100万	月¥5-10万 × 10ヶ月
LLM API + OCR API	¥20-30万	Gemini Flash + Document AI 10ヶ月分
積算士（業務委託・段階縮小）	¥120-200万	Phase1: ¥30-50万×3ヶ月 + Phase2: ¥15-25万×3ヶ月 + Phase3: ¥2-5万×4ヶ月。最終的に¥0
建設物価DB	¥5万	書籍版年間
アノテーションツール	¥0-5万	Roboflow Free tier
開発コスト合計	**¥150-250万**	**10ヶ月間**
月額運用コスト	**¥10-15万**	本番運用時

```
投資回収:
  BPO月額¥250,000 × 10社 = 月商¥250万
  → 6-10社獲得で開発費回収
  → 積算士1人雇用（年収400-600万）より安い

比較:
  積算士1人 = 年間400-600万 → 1人分の処理能力
  AI + 積算士（監修のみ）= 年間200-300万 → 4-8人分の処理能力
```

17.12 リスクと対策

リスク	影響	確率	対策
日本図面の訓練データ不足	精度が上がらない	中	BIM→2D自動生成で量産 + 確認ループで蓄積
複数図面マッチング精度不足	数量がズレる	高	確認UIで人間が修正。段階的に精度改善
パイロット企業が見つからない	実データがない	中	建設業の知人・税理士経由で紹介。無料お試し提供
積算士の監修者確保	ルールDBが作れない	中	クラウドワークス等で副業募集。月4時間の軽い関与
GPU/APIコストの増大	赤字運用	低	Gemini Flash（低コスト）+ 推論バッチ化で最適化
LLMの精度が不安定	ハルシネーション	中	CV検出結果とLLM判断のクロスバリデーション
図面品質のバラつき	低解像度・手書き図面で精度低下	高	データ拡張でノイズ耐性を訓練。最低解像度を設定

17.13 成功基準

```
Phase 1 完了基準（4ヶ月目）:
  - [ ] 日本の平面図で壁・柱・開口部をmAP 85%+で検出
  - [ ] OCRで寸法を90%+で読み取り
  - [ ] パイロット1社の実図面で「使えそう」と評価

Phase 2 完了基準（7ヶ月目）:
  - [ ] 平面図+構造図から統合BuildingGraphを生成
  - [ ] 数量ドラフトが手動積算と±20%以内
  - [ ] パイロット3社で実案件テスト開始

Phase 3 完了基準（9ヶ月目）:
  - [ ] 確認UIで人間修正が可能
  - [ ] AI+人間確認の実質精度95%+
  - [ ] 1案件の拾い出し時間: 8時間→2時間以下
  - [ ] 既存estimator.pyとの統合完了

6ヶ月運用後:
  - [ ] 実質精度98%+
  - [ ] 訓練データ3,000枚以上蓄積
  - [ ] 月10案件以上をAI拾い出しで処理
```

17.14 積算士ナレッジ吸収戦略

> **目標**: 積算士を業務委託で3-6ヶ月確保し、その間にAIが積算士のナレッジを吸収。
> 最終的にAIだけで回る体制に移行し、人件費ゼロを実現する。

Phase 1（Month 1-3）：積算士に働いてもらいながらナレッジを記録

```
積算士の全ての判断をデータとして記録する:

① 図面の読み方（視覚パターン）:
   確認UIで積算士が修正 → 修正前後のペアが訓練データに
   例: AIが「柱」と判定 → 積算士が「PS（パイプスペース）」に修正
   → 次回から同パターンをPSと判定
   蓄積ペース: 月100-200件

② 数量の拾い方（計算手順）:
   積算士に「どこから何をどう計算したか」を構造化入力してもらう
   例: 壁面積 = 通り芯間距離6m × 階高3.5m − 開口部2箇所 = 13.44m²
   蓄積ペース: 月50-100件

③ 工種分類のルール（分類知識）:
   正規化辞書 + 分類ルールの自動蓄積
   既存のCATEGORY_RULES + term_normalizationを拡張
   蓄積ペース: 月30-60件

④ 単価の判断根拠（価格知識）:
   learn_from_result()を拡張して「理由」も記録
   例: 山間部→運搬距離長い→生コン単価1.23倍
   蓄積ペース: 月100-200件

⑤ 経験則・例外判断（暗黙知）:
   月1回のヒアリングセッション + 判断ログ
   例: 「曲面壁は型枠ロス1.2倍」「地下工事は地上の1.3-1.5倍」
   蓄積ペース: 月10-20ルール
```

Phase 2（Month 4-6）：AIが半分やる

```
LLM Vision + 蓄積データで①②のドラフトを自動生成
積算士は「ゼロから拾う」→「AIの結果を確認・修正する」に変化
作業時間: 1案件3時間 → 1時間
修正内容が全て次の学習データに
担当社数: 4-8社 → 8-15社にスケール
```

Phase 3（Month 7-10）：積算士の役割を縮小

```
60-120案件分のナレッジ蓄積 + 図面解析AIの精度向上
AIが①②を85-90%自動化
積算士: 週1回の品質チェックのみ → 月4時間（¥2-5万）に縮小
あるいは完全にさよなら → スポット依頼のみ
```

3ヶ月後のナレッジDB全体像

種類	蓄積件数	活用方法
図面パターン	300-600件	物体検出モデル再訓練 / LLMプロンプト改善
計算手順	150-300件	ルールエンジンに組み込み
分類ルール	90-180件	CATEGORY_RULES + term_normalization
単価根拠	300-600件	単価推定の精度向上
経験則	30-60ルール	補正係数の自動適用
合計	**約1,000-1,700件**	**積算士の脳の80%をDB化**

コスト推移と粗利率の変化

```
Month 1-3:  積算士¥30-50万/月（フル稼働）→ 粗利率 50-60%
Month 4-6:  積算士¥15-25万/月（半日稼働）→ 粗利率 65-75%
Month 7-10: 積算士¥2-5万/月（週1品質チェック）→ 粗利率 80-85%
Month 11-:  積算士¥0（スポット依頼のみ）→ 粗利率 85-90%
```

技術的に吸収できない残り20%

```
・「この設計者の図面はいつも寸法が甘い」→ 個人の信頼度
・「この元請けとは過去にトラブルがあった」→ 人間関係
・「この現場は搬入路が狭い」→ 現地情報
・「今月は鉄筋が値上がりしそう」→ 市況の勘

→ これらは社長自身が知っている情報
→ 社長に最終確認してもらえば積算士なしでも補える
```

17.15 積算士なしルート（小規模工事特化・LLM Vision）

> 積算士を確保できない場合の代替戦略。マルチモーダルLLMに図面を直接投げる。
> 対象: 小規模民間工事（住宅リフォーム、小規模改修、外構、解体）

アプローチ

```
専用CVモデル（RT-DETR等）の訓練を待たず、
Gemini 2.5 Flash / Claude 4.5 のVision機能で図面を直接解析する。

Step 1: 図面画像をLLM Visionに投げる
  → 「この図面から部材・寸法・数量を抽出してください」
Step 2: 抽出結果を社長に確認してもらう（5-10分）
Step 3: 修正済み数量を estimator.py に渡す（③④⑤⑥は自動）
```

精度（図面の種類別）

図面の種類	LLM Vision精度	社長確認後の精度
小規模（住宅リフォーム・改修）	70-80%	±10-15%
中規模（3階建て事務所）	50-60%	±15-20%
大規模（マンション・公共施設）	30-40%	対象外（積算士ありルートを使用）

積算士ありルート vs LLM Visionルートの比較

	積算士ありルート（17.14）	LLM Visionルート（17.15）
対象工事	全規模	小規模のみ
最終精度	±5%	±10-15%（社長確認後）
人件費	¥30-50万/月（Phase 1）→ ¥0	¥0（初日から）
開始まで	積算士確保が必要	即日
開発期間	4-6週間	2ヶ月

推奨: 両ルートの併用

```
案件規模で自動振り分け:
  小規模民間工事（500万円以下）→ LLM Visionルート（積算士不要）
  中大規模工事（500万円超）      → 積算士ありルート（Phase 1-3でAI移行）
→ 最初から両方提供し、幅広い案件に対応
```